

Entrega Final – Aplicaciones Web 2

Proyecto: UrbanWear

Alumno: Nelson Sánchez **Carrera:** Analista de Sistemas y Desarrollo de Software **Materia:** Aplicaciones Web 2
Institución: IES Siglo 21

Repositorio GitHub

<https://github.com/NelCount/urbanwear-json-aw2>

Video Explicativo

<https://drive.google.com/drive/folders/1KqbHWv2azuOoOjfYrucq-K5FxnJRZk1x?usp=sharing>

Descripción General del Proyecto

UrbanWear es una aplicación Full Stack de comercio electrónico desarrollada para la materia Aplicaciones Web 2. La aplicación permite gestionar productos, usuarios y ventas mediante una API REST propia, incorporando además un sistema de autenticación basado en JWT y una integración con Mercado Pago Checkout Pro en entorno de pruebas.

Las principales funcionalidades implementadas son:

- Registro e inicio de sesión de usuarios.
- Gestión de productos desde MongoDB.
- Carrito de compras persistente mediante LocalStorage.
- Generación de órdenes de compra.
- Integración con Mercado Pago Checkout Pro.
- Persistencia de datos utilizando MongoDB Compass.

Tecnologías Utilizadas

Frontend

- HTML5
- TailwindCSS
- JavaScript (Vanilla JS)
- LocalStorage

Backend

- Node.js
- Express.js
- JWT (JSON Web Tokens)
- bcryptjs
- dotenv

Base de Datos

- MongoDB Atlas
- Mongoose

Integraciones Externas

- Mercado Pago Checkout Pro (Sandbox)

Dependencias Principales

```
express
mongoose
bcryptjs
jsonwebtoken
dotenv
mercadopago
cors
nodemon
```

Arquitectura Utilizada

```
Frontend (HTML + JavaScript + TailwindCSS)
      ↓
API REST (Node.js + Express.js)
      ↓
MongoDB Atlas (Mongoose)
      ↓
Mercado Pago Checkout Pro
```

El proyecto fue organizado utilizando una arquitectura desacoplada basada en:

- Frontend y Backend separados.
- Rutas (Routes).
- Modelos (Models).
- Middlewares.
- Variables de entorno (.env).

- Persistencia mediante MongoDB Atlas.

Funcionalidades Implementadas

Usuarios

- Registro de usuarios.
- Inicio de sesión.
- Encriptación de contraseñas mediante bcryptjs.
- Autenticación mediante JWT.
- Persistencia de sesión mediante LocalStorage.

Productos

- Listado dinámico de productos.
- Consulta individual de productos.
- Creación y actualización de productos.
- Filtrado por categorías.
- Imágenes dinámicas.

Carrito de Compras

- Agregar productos al carrito.
- Modificar cantidades.
- Eliminar productos.
- Cálculo automático del total.
- Persistencia mediante LocalStorage.

Ventas

- Generación de órdenes de compra.
- Asociación de compras al usuario autenticado.
- Persistencia de ventas en MongoDB.

Pagos

- Integración con Mercado Pago Checkout Pro.
- Generación de preferencias de pago.
- Renderizado del Checkout oficial de Mercado Pago en entorno Sandbox.

Operaciones CRUD Demostradas

CREATE

- Registro de usuarios mediante `POST /api/v1/users/register`.
- Creación de productos mediante `POST /api/v1/products/create`.

READ

- Consulta de productos mediante `GET /api/v1/products/all`.
- Consulta de usuarios mediante `GET /api/v1/users/all`.
- Consulta de ventas mediante `GET /api/v1/sales/all`.

UPDATE

- Actualización de productos mediante `PUT /api/v1/products/update/:id`.

DELETE

- Eliminación de usuarios y ventas mediante los endpoints correspondientes.

Mejoras Implementadas Durante el Desarrollo

- Migración de persistencia desde archivos JSON hacia MongoDB Atlas.
- Implementación de autenticación basada en JWT.
- Incorporación de bcryptjs para el hash de contraseñas.
- Implementación de LocalStorage para la persistencia del carrito.
- Integración de Mercado Pago Checkout Pro en entorno Sandbox.
- Mejora de la interfaz mediante TailwindCSS.
- Carga dinámica de imágenes de productos.
- Gestión de cantidades y eliminación de productos en el carrito.
- Separación del proyecto en frontend y backend utilizando una arquitectura desacoplada.

Principales Aprendizajes Obtenidos

Durante el desarrollo del proyecto se adquirieron conocimientos prácticos sobre:

- Construcción de APIs REST utilizando Express.js.
- Modelado y persistencia de datos con MongoDB y Mongoose.
- Implementación de sistemas de autenticación mediante JWT.
- Protección de contraseñas mediante hashing.
- Consumo e integración de APIs externas.
- Organización de proyectos Full Stack mediante una arquitectura por capas.
- Implementación de un flujo completo de compra utilizando Mercado Pago Checkout Pro.

Mejoras Futuras

- Implementar un panel de administración.
- Incorporar gestión automática de stock.
- Agregar historial de compras por usuario.

- Implementar Webhooks de Mercado Pago.
 - Incorporar recuperación de contraseña.
 - Desarrollar un dashboard de métricas y ventas.
 - Realizar el despliegue de la aplicación en producción.
-

Conclusión

UrbanWear permitió aplicar de manera integral los conceptos desarrollados durante la materia Aplicaciones Web 2, incluyendo el desarrollo de APIs REST, la utilización de bases de datos NoSQL, la autenticación mediante JWT, la persistencia de datos y la integración de servicios externos.

El proyecto evolucionó desde una implementación inicial basada en archivos JSON hasta convertirse en una aplicación Full Stack completa, incorporando tecnologías y herramientas utilizadas actualmente en el desarrollo profesional de aplicaciones web.